2021 International Conference on Data Mining Workshops (ICDMW) | 978-1-6654-2427-1/21/$31.00 ©2021 IEEE | DOI: 10.1109/ICDMW53433.2021.00099

Disjoint-CNN for Multivariate Time Series Classification

Seyed Navid Mohammadi Foumani
Faculty of Information Technology
Monash University
Melbourne, Australia
navid.foumani@monash.edu

Chang Wei Tan
Faculty of Information Technology
Monash University
Melbourne, Australia
chang.tan@monash.edu

Mahsa Salehi
Faculty of Information Technology
Monash University
Melbourne, Australia
mahsa.salehi@monash.edu

***Abstract*—Time series classification algorithms have been mainly dominated by non-deep learning models. Deep learning for Multivariate Time Series Classification (MTSC) has gained huge interest in recent years. Most state of the art deep learning methods are convolutional-based where 1-dimensional (1D) convolutions are used to extract features from the 2-dimensional time series. This study shows that factorization of 1D convolution filters into disjoint temporal and spatial components yields significant accuracy improvements with almost no additional computational cost. Based on our study on disjoint temporal-spatial filters, we have designed a novel filter block called "1+1D", which emphasizes the interaction between dimensions to improve the model performance of the convolution based on deep learning MTSC models. We also proposed a new and effective MTSC method called Disjoint-CNN using our proposed 1+1D filter blocks and through our extensive experiments show that our model (called Disjoint-CNN) outperforms the state-of-the-art MTSC models on 26 datasets in the UEA Multivariate time series archive, achieving the highest average rank among 9 MTSC benchmark models.**

***Index Terms*—convolutional neural networks, temporal-spatial filters, multivariate time series classification.**

I. Introduction

Time series classification (TSC) is a fast growing field with hundreds of papers being published every year [1]–[11]. Each of them being more accurate than its predecessors when benchmarked on a set of benchmark datasets [3], [4]. Most of the work in TSC has been focusing on univariate time series where only one variable (channel) is recorded over time. On the other hand, multivariate TSC (MTSC) [1], [11], where the time series has more than one variables, received relatively less attention in the community compared to univariate TSC. This is reflected in the size of the benchmark datasets where the univariate archive [4] is 5 times larger than the multivariate archive [3] with 128 and 26 datasets respectively. MTSC received less attention mainly due to the fact that multivariate time series are more complex to study and there are many factors such as the interaction between variables, that would affect the performance of a MTSC classification algorithm.

Although the interaction between each channel is important, most existing state-of-the-art MTSC algorithms do not consider them [1]. They simply extract features independently from each channel [12]–[14] or from random subset of all channels [15] and train a classifier using the extracted features.

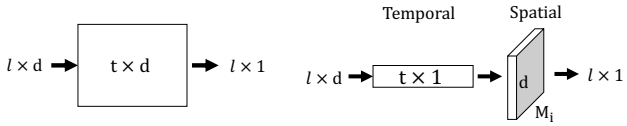


Fig. 1: **1D vs 1+1D convolution**. This figure shows for the simplified setting where a $l \times d$ input is fed into the convolution layer and generate a $l \times 1$ output, l denotes the series length and d is the number of input channels. a) 1D convolution, which applies a $t \times d$ filter where t shows the temporal extent ($t < l$) . b) Our proposed 1+1D convolution block, which splits the computation into a disjoint temporal and spatial convolutions. We set the number of spatial convolutions M_i so that the number of parameters in 1+1D matches that of a typical 1D convolutions.

Univariate TSC algorithms are also being adapted to MTSC by training an ensemble of univariate TSC algorithms built on each channel independently [1], [16].

Convolutional Neural Networks (CNNs) are highly successful in many applications, especially in computer vision and speech recognition, often outperforming previous state-of-the-art methods [17]–[19]. They are also well suited for end-to-end learning tasks by learning from the raw data without any prior feature selection. Besides, CNNs also scale well to large datasets and is able to exploit the hierarchical structure in the data. There have been limited studies done on using CNNs for MTSC [1], [7], [8], [16], [20], [21].

Existing models consider a time series as a 1-Dimensional (1D) image and employ 1D convolution operations to extract features from the multivariate time series. Similar to other state-of-the-art MTSC algorithms, they do not consider the importance of the interaction between channels. Sometimes univariate CNN models are also used on the multivariate time series input data by considering them as multiple independent univariate series [16], similar to the work in [1]. In this paper, we challenge this view and introduce a convolution block, called 1+1D that emphasizes on the interaction between the input channels. Our 1+1D block explicitly factorizes 1D convolution into two unmixed and successive operations, i.e., 1D

978-1-6654-2427-1/21/$31.00 ©2021 IEEE
DOI 10.1109/ICDMW53433.2021.00099

Authorized licensed use limited to: UNIVERSIDAD DE GRANADA. Downloaded on March 16,2025 at 13:30:20 UTC from IEEE Xplore. Restrictions apply.

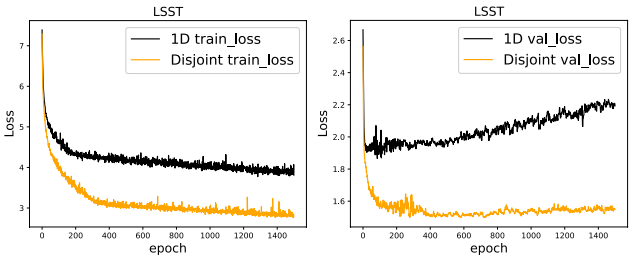


Fig. 2: Comparing the training (left) and validation (right) loss of our proposed 1+1D convolution block and typical 1D convolution block using the LSST dataset from the UEA MTSC archive [3].

temporal convolution per channel and 1D spatial convolution that learns the interaction between the channels through the extracted features from the temporal convolution. 1+1D is also closely related to the Pseudo-3D network (P3D) [22] and Spatiotemporal convolutions [23], which obtain promising results in 3D data in computer vision by factorizing the 3D convolutions to 2D and 1D convolutions, respectively.

Fig. 1 illustrates the simplified setting of this decomposition and compares to the traditional 1D convolution used in most existing CNN models. There are two main benefits of such decomposition. First is that we can use an additional nonlinear activation function between these two operations that provides additional nonlinear representation to the model. The use of additional nonlinear activation doubles the number of nonlinear functions for the model, allowing the model to extract more complex functions with less parameters, while keeping the number of the parameters approximately the same as the typical 1D convolution operation. Section III-B2 shows how we keep the number of parameters the same.

Second, such decomposition facilitates the optimization process in our deep neural network model. Simply put, we find that compared to typical 1D filters where temporal and spatial filters are jointly intertwined, the 1+1D blocks are easier to optimize, yielding lower test and training loss in practice. Fig. 2 shows the training and validation errors for 1D and 1+1D convolution for the LSST dataset from the UEA MTSC archive [3]. We consider the same number of layers and parameters for both networks (for more details see Section III-B2). It can be seen that, for the same number of layers and parameters, 1+1D convolution results in lower training and validation errors compared to the typical 1D convolution. We further demonstrate that our 1+1D block can improve existing CNN based models by simply replacing existing 1D convolutions with 1+1D convolution. Notably, the performance of the model heavily depends on the underlying network. In Section IV, we show how the performance of baseline CNN models, Fully Convolutional Neural Network (FCN) and Residual Network (ResNet) [8] can be improved using our proposed 1+1D.

By decomposing the 1D convolutions to disjoint temporal and spatial convolutions, we propose a new MTSC algorithm called Disjoint-CNN that is more accurate that the current state of the art algorithms [1] on 26 MTSC datasets of the UEA repository [3]. The rest of the paper is organized as follows: We give a brief background information and describe some of the related work in Section II. Section III introduces the problem and describes the proposed Disjoint-CNN model in details. Then Section IV describes the implementation details and evaluates our proposed model. Finally, we conclude our paper in Section V and give some possible directions for future work.

II. Related work

A recent survey on MTSC [1] has provided an extensive survey and benchmark for MTSC. The state-of-the-art MTSC algorithms can be divided into seven main categories – distance-based, interval-based, shapelet-based, feature-based, dictionary-based, ensemble of univariate models and deep learning models. In this section, we aim to briefly discuss some of the existing state-of-the-art MTSC algorithms and we refer interested readers to the survey paper [1] for more detailed discussion of MTSC algorithms. We divide this section into two parts, subsection II-A describes non-deep learning based approaches while subsection II-B describes the deep learning models.

A. non-deep learning approaches for MTSC

Distance based approaches rely on the shape of the time series. They consist of using a 1-nearest neighbour (1-NN) classifier with a distance function [1], [5]. There are many distance functions defined for univariate time series [5], however only Dynamic Time Warping (DTW) has been shown to be effective for multivariate time series [1], [3], [24]. Typically, DTW is computed between a pair of univariate time series using dynamic programming. It computes a distance matrix where each cell represents the Euclidean distance of two time points between the time series. We refer interested readers to the work [24] for a more detail explanation of multivariate DTW.

Interval-based algorithms are known to be interpretable for TSC [15], [25]. They extract features from intervals of a time series and typically use a tree-based classifier. The Canonical Interval Forest (CIF) is the most accurate interval-based algorithm for both univariate and multivariate TSC [15]. CIF is an ensemble of time series tree [25] that is built using 3 simple summary statistics and 22 Canonical Time Series Characteristics (Catch22) features [26] extracted from random intervals of the time series. CIF handles multivariate time series by randomly selecting a dimension for each interval.

Shapelets [27] are discriminatory sub-series of the time series which are highly interpretable compared to many other TSC algorithms. The Generalised Random Shapelet Forest (gRSF) is a tree-based ensemble for both univariate and mutivariate TSC [13]. gRSF extends the original Time Series Shapelets algorithm [27] that uses a single shapelet-based decision tree to a forest, where each tree randomly selects a set of shapelets from a randomly chosen dimension at each

Authorized licensed use limited to: UNIVERSIDAD DE GRANADA. Downloaded on March 16,2025 at 13:30:20 UTC from IEEE Xplore. Restrictions apply.

node of the tree. The algorithm employs bagging, similar to a Random Forest [28] to create diversity in the classifiers.

Different from interval-based and shapelet-based algorithms, feature-based algorithms typically extract features from the whole time series using some bespoke feature operations. SEQL [29] is a symbolic sequence learning algorithm that was originally designed as a binary classifier for sequence data such as DNA or text. The algorithm efficiently traverses a large feature space by employing a branch-and-bound feature search strategy and selects the most discriminative subsequences for a linear model. SEQL was adapted for time series data by discretising the time series through Symbolic Aggregate Approximation (SAX) [30], [31], creating an interpretable TSC algorithm. The Multiple Representation Sequence Learner, MrSEQL [14] extends the previous work by building and ensemble and integrating the Symbolic Fourier Approximation (SFA) [32] to create multiple representation of the time series. MrSEQL was not designed for MTSC but has been adapted to processing multivariate time series where the channels are processed sequentially and appended to one another [1].

The Random Convolutional Kernel Transform (ROCKET) [6] is a scalable and accurate TSC algorithm. It was found to be the most accurate and scalable MTSC algorithm in a recent survey [1]. By default, it applies 10,000 random convolution kernels to each time series. Two features, maximum and proportion of positive values are extracted from the convolution outputs, forming a 20,000 feature vector for each time series. Then a linear classifier is used for classification. ROCKET was originally designed for univariate TSC, however it has been adapted easily for MTSC by assigning channels to each kernel randomly, where the convolution operation is applied across all the channels [1], [6].

Dictionary-based algorithms convert a time series into a bag of words (patterns) which can then be passed onto a classification algorithm. The core process involves sliding a window across the time series and discretising each window by applying SAX [31] or SFA [32]. One of the state-of-the-art univariate TSC algorithms, Word Extraction for Time Series Classification (WEASEL) was extended to MTSC using Multivariate Unsupervised Symbols and Derivatives (MUSE) also known as WEASEL+MUSE [1], [12]. WEASEL+MUSE extracts unigrams and bigrams from all the channels of the raw and derivatives of the time series using SFA. It then applies a χ^2 test for features (words) selection. Then the final set of words are passed to a logistic regression classifier [12].

In contrast to the previously mentioned bespoke MTSC algorithms, one of the most straightforward way of adapting univariate TSC algorithms to MTSC is to create a single model per dimension independently and ensemble of them to create a probability distribution for classification [1]. The work in [1] has ensemble some of the state-of-the-art TSC algorithms. This includes the Hierarchical Vote Collective of Transformation-based Ensembles (HIVE-COTE v1.0) [33], and its components, Shapelet Transform Classifier (STC) [34], TSF [25], Contractable Bag of Symbolic Fourier Approximation Symbols (CBOSS) [35] and Random Interval Spectral Ensemble (RISE) [36].

B. Deep learning for MTSC

Deep learning models, especially Convolutional Neural Networks (CNNs) have been successful and popular in various image applications. Recently they have being slowly extended to univariate time series data [7], [8], [11], [16]. Unlike most algorithms discussed in Section II-A, deep learning models handle multivariate time series naturally and have shown promising results for MTSC [1], [7], [8]. In this section, we briefly discuss some deep learning models for MTSC and refer interested readers to two recent surveys on deep learning for MTSC [1], [8].

CNNs are the most popular architectures for MTSC due to their ability to extract latent features from the time series data efficiently. A Multi-Channel Deep Convolutional Neural Network (MC-DCNN) [16] was proposed and evaluated on two multivariate datasets. MC-DCNN applies three convolution layers independently to each dimension. The output from each dimension is concatenated and passed into a fully connected layer which is then fed to a final softmax classifier for classification.

Fully Convolutional Neural Network (FCN) is a simple convolutional network that does not contain any pooling layers. It was first proposed in [37] and later being evaluated in [8]. It contains three convolution blocks where each block contains a convolution layer followed by batch normalisation and ReLU activation. Each block uses 128, 256, 128 filters with 8, 5, 3 filter length respectively. The output from the last convolution block is averaged with a Global Average Pooling (GAP) layer and passed to a final softmax classifier.

Similar to FCN, the Residual Network (ResNet) was also proposed in [37] and evaluated in [8]. ResNet is one of the deepest architectures for MTSC (and TSC in general) containing three residual blocks followed by a GAP layer and a softmax classifier. It uses residual connections between each block to reduce the vanishing gradient effect in deep learning models. The structure of each residual block is similar to the FCN architecture containing 3 convolution layers followed by batch normalisation and ReLU activation. Each convolution layer uses 64 filters with 8, 5, 3 filter length respectively. ResNet was found to be one of the most accurate deep learning TSC architecture on 85 univariate TSC datasets [2], [8]. It was also proven to be an accurate deep learning model for MTSC [1], [8].

Due to the highly favorable performance of the FCN model, various extensions like MLSTM-FCN [11] and TapNet [21] were proposed later to deal with time-series data. MLSTM-FCN model is proposed based on the univariate LSTM-FCN model [38]. Like the LSTM-FCN, the multivariate version comprises LSTM blocks and fully convolutional blocks for extracting features from input series. The squeeze and excite block is also added to the FCN block, and can execute a form of self-attention on the output feature maps of previous layers [11]. Time series attentional prototype network (TapNet) is also proposed for multivariate time series classification [21].

Authorized licensed use limited to: UNIVERSIDAD DE GRANADA. Downloaded on March 16,2025 at 13:30:20 UTC from IEEE Xplore. Restrictions apply.

TapNet designed based on traditional distance-based and deep learning approaches to leverage the strengths of both of them. This model has three main parts. First, the model applies random dimension permutation to the multivariate input time series to group randomly selected dimensions and facilitate learning from combinations of dimensions. Then, similar to MLSTM-FCN, each group is fed to the LSTM and FCN blocks for extracting features. Finally, low-dimensional outputs from these feature extractors are learned based on their distance by attention prototype network.

InceptionTime is the current state-of-the-art deep learning model for both univariate TSC and MTSC [1], [7]. InceptionTime is an ensemble of five randomly initialised InceptionNetwork modules consists of two blocks of inception modules. Each inception module first reduces the dimensionality of a multivariate time series using a bottleneck layer with length and stride 1 while maintaining the same length. Then, 1D convolution of different lengths are applied to the output of the bottleneck layer to extract patterns at different sizes. Besides, a Max Pooling layer followed by a bottleneck layer are also applied to the original time series to increase the robustness of the model to small perturbations. The output from the convolution and max pooling layers are stacked to form a new multivariate time series which is then passed to the next layer. Residual connections are also used between each inception block to reduce the vanishing gradient effect. The output of the second inception block is passed to a GAP layer before feeding into a softmax classifier.

III. Proposed Model

This section gives a more precise definition of the problem being addressed in this paper and introduces our Disjoint-CNN model step by step.

A. Problem Definition

Given a time series dataset X with m instances, $X = \{\mathbf{x_1}, \mathbf{x_2}, ..., \mathbf{x_m}\}$, where $\mathbf{x_t} = \{x^1, x^2, ..., x^N\}$ is a d-dimensional time series and N is the length of time series, $\mathbf{x_t} \in \mathbb{R}^{N \times d}$, and the set of relevant response labels $Y = \{y_1, y_2, ..., y_m\}$, $y_t \in \{1, ..., c\}$ and c is the number of classes. The aim is to train a neural network classifier to map set X to Y. Using the classifier, we estimate the probability of labels C as follows:

$$P(C)_t = f(\theta, \mathbf{x_t}) \tag{1}$$

where:

- $P(C)_t$ is the vector of response probability for a given $\mathbf{x_t}$, and C is the response class
- f the neural network function
- θ all parameters of model (all weights and biases)

A good choice to estimate multinomial (including binomial), categorical dependent variables in neural networks is the softmax activation function in the final layer of the neural network. The softmax function estimates probabilities for all of the dependent classes such that they always sum to 1 across all classes. If the softmax activation function is used in the final layer of the neural network, solving the problem defined above requires finding the parameters θ of a given neural network architecture that minimizes the cross-entropy loss function, which is defined as follows:

$$L_i = -\sum_{j=1}^{c} y_{j,i} log(\hat{y}_{j,i}) \tag{2}$$

where:

- L_i is the loss for training observation i
- i is the training observation instance or time point
- $y_{j,i}$ is the class label for class j (of c classes) and training observation i, $y_{j,i} \in \{0, 1\}$
- $\hat{y}_{j,i}$ is the neural network prediction for class j and observation i, $\hat{y}_{j,i} \in \{0, 1\}$

The cross-entropy loss function penalizes low predictions for the target class while ignoring predictions for other classes due to multiplication of the loss by $y_{j,i}$ which is 1 for the target class and 0 for other classes.

B. 1+1D Convolution blocks

The provided related work highlights the success of CNNs in various time series classification tasks. The CNN architecture is well suited for learning complex patterns in the temporal domain. They incorporate two main ideas for different tasks on inputs such as image or time-series data. These types of data often have hierarchical structures, and CNN's can learn local non-linear patterns through the convolutions filters and activation functions and represent higher-level patterns as a combination of lower-level patterns. Also, by applying the pooling layer, they can create a coarser intermediate feature representation that enables CNNs to learn position-invariant features in the input data. Therefore, we look closely at the 1 Dimensional (1D) filters on univariate and multivariate time series input and then define the 1+1D convolution blocks and their benefits compared to 1D convolutions.

1) 1D convolution: Fig. 3 represents the 1D convolution on univariate and multivariate time series data. As shown in Fig. 3 (a) when the input is a univariate time series, the shape of the filters is one-dimensional $F \in R^t$. These kinds of convolutions are also used with the intention of applying the univariate CNN models to the multivariate time series input data by considering them as multiple independent univariate series [16]. As this convolution only extracts temporal information, it can be called a Temporal convolution.

However, when the input is a multivariate time series Fig3 (b) the filter shape is $F \in R^{d \times t}$ in which d is equal to the number of input dimensions and t is the length of the filter that is defined by the user. Simply put, when we define a 1D filter for 2D inputs we just set the length of the filter, and the width of the filter is always set equal to the number of input dimensions. As depicted in Fig3 (b) for the 10 dimensional time series, the size of 1D filter is $R^{10 \times 3}$.

2) 1+1D convolution blocks: In this work, we hypothesized that 1D convolution on multivariate time series may be more conveniently approximated by a temporal convolution followed by a spatial convolution. We thus design a convolution

Authorized licensed use limited to: UNIVERSIDAD DE GRANADA. Downloaded on March 16,2025 at 13:30:20 UTC from IEEE Xplore. Restrictions apply.



Fig. 3: 1D filters with size=3 (a) Univariate time series with 1D convolution (b) Multivariate time series with 1D convolution

block named 1+1D, where we replace the 1D filters with 1+1D block consisting disjoint temporal and spatial filters. In this process, we replace N_i 1D convolution filters of size $t \times d$ with a 1+1D blocks consisting M_i temporal convolution filters of size $t \times 1$ and N_i convolution filters of $M_i \times d$. The hyperparameter M_i specifies the intermediate subspace's dimensionality where the input signals are projected between temporal and spatial filters.

Therefore, we choose M_i as follows:

$$M_i = \lfloor N_i(t \times d)/t + N_i \times d \rfloor \tag{3}$$

Thus, the number of parameters of the 1D convolution is approximately equal to the 1+1D block. The architecture of the 1+1D block is depicted in Fig. 4. Compared to 1D convolutions, our 1+1D decomposition offers two advantages. First, despite not changing the number of parameters, it doubles the number of nonlinearities in the network due to the additional activation function between the temporal and spatial convolution in each block. Increasing the number of nonlinearities increases the complexity of functions.

The second benefit is that factorizing the 1D convolution into separate temporal and spatial components makes the optimization process easier. This is manifested in the lower training and validation loss of the `LSST` dataset compared to the 1D convolution with the same capacity, shown in Fig. 2. Similar architectures are used for comparing the two blocks (Fig. 2). For simplification purposes, we used a single layer CNN in which the filter length is equal to 8 ($t = 8$). The number of 1D filters is set to 32; since the `LSST` dataset has 6 dimensions ($d = 6$), M_i is set to 7 based on equation 3 in order to have approximately the same number of parameters for 1D and 1+1D convolution blocks. More precisely, the 1D convolution has 1568 parameters ($32 \times 8 \times 6$ for weights and 32 for bias). Also, the 1+1D has 1439 parameters, from which 63 parameters are dedicated to Temporal convolutions ($7 \times 8 \times 1$ for weights and 7 for bias) and 1376 parameters to spatial convolutions ($32 \times 6 \times 7$ for weights and 32 for bias). It can be seen that for the roughly same number of layers and parameters, the 1+1D convolution yields not only a lower validation loss but also a lower training loss compared to the 1D type. This is an indication that optimization becomes easier when temporal-spatial filters are factorized. Furthermore, as shown in Fig. 2, the 1D convolution begins to overfit this dataset with very small epochs while 1+1D does not. This shows that 1+1D is more suitable than the 1D convolution on this UEA MTSC dataset. The other datasets in the UEA MTSC repository also show similar trends for training and validation loss, but we are not showing them due to the space limitation.

C. Disjoint-CNN Architecture Overview

Fig. 4 depicts the overall architecture of our proposed MTSC model, Disjoint-CNN that uses four of our proposed 1+1D blocks. As depicted in Fig. 4, temporal filters were applied to the input data as a first step in 1+1D block. In this step, the model extracts temporal patterns that occur in the input signals. Next, the output of temporal filtering are fed into the spatial filters after applying the batch normalization (BN) and Exponential Linear Units (ELUs) [39] activation function which is defined as follows:

$$f(x) = \begin{cases} x & x > 0 \\ e^x - 1 & x \leq 0 \end{cases} \tag{4}$$

Before applying any activation functions, we batch-normalize the intermediate temporal and spatial filtering outputs in order to make these representations closer to the normal distribution and ease gradient optimization [40]. Finally, just before the fully connected layer, max-pooling and global average pooling (GAP) are applied to the output of the last layer's ELU activation function, which gives a more translation-invariant model.

IV. Experimental Results

In this section, we evaluate the performance of our Disjoint-CNN on MTSC repository and compare with the state-of-the-art models. Further experiments are also conducted to assess the effect of our Disjoint-CNN's parameters and components on the performance and to analyze the results. All of our experiments are conducted using the TensorFlow framework in Python on two Tesla V100 GPU with 16GB memory. To promote reproducibility, we have provided our source code and more experimental results online [1].

Our experiments are divided into three parts. First, we show that our Disjoint-CNN outperforms existing CNN models on the UEA MTSC repository [3]. Then we compare Disjoint-CNN with state of the art models (consisting of both deep learning and non-deep learning categories), using results from [1]. Finally, we do an ablation study to investigate the effect of each component of 1+1D and Disjoint-CNN on the performance of our model.

A. Datasets

We evaluate the proposed method on the 26 of the 30 MTSC archive problems where the input time series are all of equal length [3]. The archive consists of real-world multivariate time series data from different applications such as Human Activity Recognition, Motion classification, and ECG/EEG signal classification. The number of dimensions ranges from

[1] https://github.com/NavidFoumani/Disjoint-CNN

Authorized licensed use limited to: UNIVERSIDAD DE GRANADA. Downloaded on March 16,2025 at 13:30:20 UTC from IEEE Xplore. Restrictions apply.

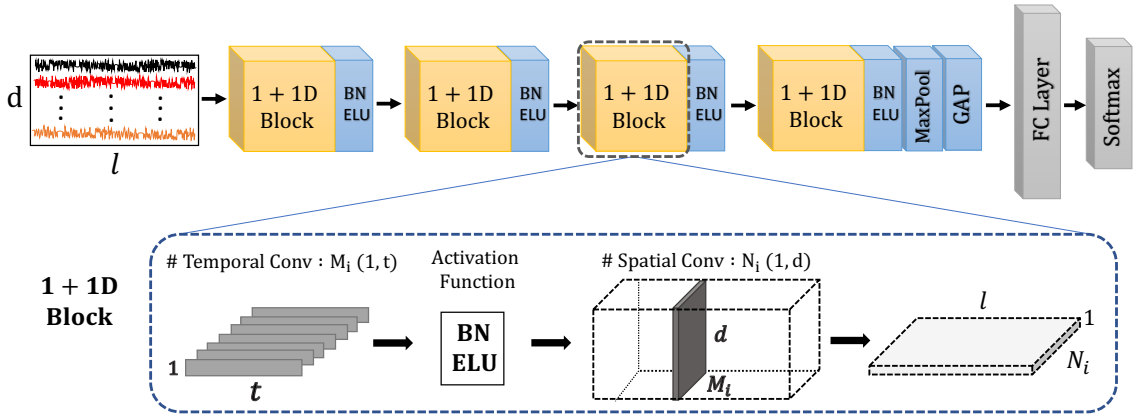


Fig. 4: Overall Architecture of the Disjoint-CNN Model and 1+1D Block

TABLE I: Summary of the 26 UEA MTSC datasets used for experiments

Problem	Short-form	Train	Test	Dim	Length	Classes
ArticularyWordRecognition	AWR	275	300	9	144	25
AtrialFibrillation	AF	15	15	2	640	3
BasicMotions	BM	40	40	6	100	4
Cricket	CR	108	72	6	1197	12
DuckDuckGeese	DDG	50	50	1345	270	5
EigenWorms	EW	128	131	6	17984	5
Epilepsy	EP	137	138	3	206	4
EthanolConcentration	EC	261	263	3	1751	4
ERing	ER	30	270	4	65	6
FaceDetection	FD	5890	3524	144	62	2
FingerMovements	FM	316	100	28	50	2
HandMovementDirection	HMD	160	74	10	400	4
Handwriting	HW	150	850	3	152	26
Heartbeat	HB	204	205	61	405	2
Libras	LIB	180	180	2	45	15
LSST	LSST	2459	2466	6	36	14
MotorImagery	MI	278	100	64	3000	2
NATOPS	NATO	180	180	24	51	6
PenDigits	PD	7494	3498	2	8	10
PEMS-SF	PEMS	267	173	963	144	7
PhonemeSpectra	PS	3315	3353	11	217	39
RacketSports	RS	151	152	6	30	4
SelfRegulationSCP1	SRS1	268	293	6	896	2
SelfRegulationSCP2	SRS2	200	180	7	1152	2
StandWalkJump	SWJ	12	15	4	2500	3
UWaveGestureLibrary	UW	120	320	3	315	8

two dimensions to 1345 dimensions. The length of the time series ranges from 8 to 17,984. The datasets also have a train size ranging from 12 to 5890. The summary of datasets is provided in Table I [2].

B. Evaluation procedure

In order to understand what a good solution (i.e., model) looks like to the problem, the impact of the various prediction outcomes needs to be understood and evaluated. We use the classification accuracy as the overall metric to compare different models. For comparing our model with convolution-based models, we use the default train and test sets provided in the repository [3]. Then, to benchmark our model with the state-of-the-art MTSC models, we follow the same procedure as [1] by running the experiments on the same thirty re-samples datasets and use the average classification accuracy for the comparison.

Then we rank each model based on its classification accuracy per dataset. The most accurate model is assigned a rank of 1 and the worse performing model is assigned the highest rank. The average ranking is taken in case of ties. We then compute the average rank for each model across the whole 26 MTSC datasets. This gives a direct general assessment of all the models: the lowest rank corresponds to the method that, on average the most accurate. The average ranking for each model is presented in the form of critical difference diagram [41], where models in the same clique (the black bar in the diagram) are not statistically significant. For the statistical test, we used the Wilcoxon signed-rank test with Holm correction as the post hoc test to the Friedman test [41].

C. Parameter setting

Adam optimization algorithm [42] is used simultaneously with an early stopping method based on validation lost. We set the default value for the number of temporal and spatial filters of 1+1D to 128 and set the length of the temporal filters to 8, 5, 5, and 3. The width of the first spatial convolutions are set equal to the input dimensions. For the rest of the layers, the number of the previous layer's output channels - which is equal to 128 - is used as the width for spatial convolutions. Similar to [40], batch normalization was applied before the activation function to intermediate outputs of the network to modify the mean and variance to 0 and 1, respectively. This normalization facilitates the optimization by keeping the inputs of hidden layers closer to normal distribution during the training. Finally,

[2]Datasets are available at http://www.timeseriesclassification.com

Authorized licensed use limited to: UNIVERSIDAD DE GRANADA. Downloaded on March 16,2025 at 13:30:20 UTC from IEEE Xplore. Restrictions apply.

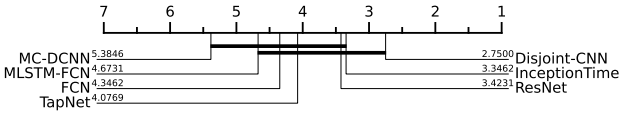


Fig. 5: Critical difference diagram for CNN-based models on MTSC benchmarking archive [3]

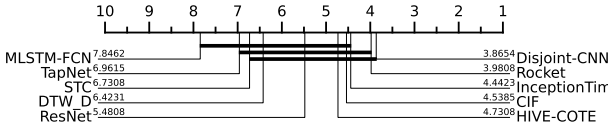


Fig. 6: Critical difference diagram for top 10 classifiers on the re-sampled MTSC benchmarking archive [1]

max-pooling and global average pooling are applied to the outputs of the activation function. The pooling create a coarser intermediate feature representation that enables CNNs to learn position invariant features in the input data.

D. Comparing with CNN-based Model

We compare our Disjoint-CNN with the following CNN models for MTSC:

1) **MC-DCNN**: Multi-channel Deep Convolutional Neural Network is a pioneer model which used CNN for MTSC. This model used only temporal convolutions to extract features from input series [16].
2) **FCN**: Fully Convolutional Neural network is one of the most accurate deep neural networks for MTSC [8].
3) **ResNet**: Residual Network is also one of most accurate deep neural networks for both univariate TSC and MTSC [8].
4) **MLSTM-FCN**: Multivariate long-short term Memory FCN used LSTM and CNN to model the sequential information in the time series [11].
5) **TapNet**: Similar to MLSTM-FCN, used recurrent neural network and CNN to capture the feature in input series. It also applies distance-based models to the neural networks' outputs to leverage the advantage of both models [21].
6) **Inception time**: The most accurate deep learning univariate TSC and MTSC algorithm to date. [1], [7].

Fig. 5 shows the critical difference diagram of the CNN-based models on default train and test sets of the UEA repository. The critical diagram demonstrates that the Disjoint-CNN with an average rank of 2.75 is the most accurate CNN-based model among all other CNN-based models. This is followed by InceptionTime and ResNet, with 3.34 and 3.42 ranks respectively. It is important to note that although our Disjoint-CNN model has the best overall ranking, it is not significantly accurate than most of the models. This is shown in the Fig. 5 where all the models, except MC-DCNN are in the same cliques. The MC-DCNN model uses only the temporal convolution to extract features from the input series. Thus, having the worse performance among all CNN-based models indicates that using temporal convolution alone is not sufficient to extract informative features from the time series compared to 1D and 1+1D convolutions. Similar results are observed in our ablation study in Section IV-F where we investigate the importance of each temporal and spatial convolution separately and the factorization of them altogether.

E. Benchmark against MTSC state of the art models

Given that our Disjoint-CNN model has the best performance compared to all the other CNN-based models, we now proceed to benchmark it against the state-of-the-art MTSC models, i.e., both deep learning and non-deep learning models.

Fig. 6 depicts the critical difference for the top 10 classifiers (including our Disjoint-CNN) on thirty re-sampled MTSC benchmarking datasets [1]; Table II presents the mean classification accuracy and the average rank of each method on UEA repository.

The highest accuracy for each dataset is highlighted in bold. Overall, our approach outperforms all the other baseline approaches in terms of average rank. Disjoint-CNN achieves 3.86, which is slightly better than ROCKET. and significantly better than other models like Inception Time and HIVE-COTE with an average rank of 4.44 and 4.73, respectively.

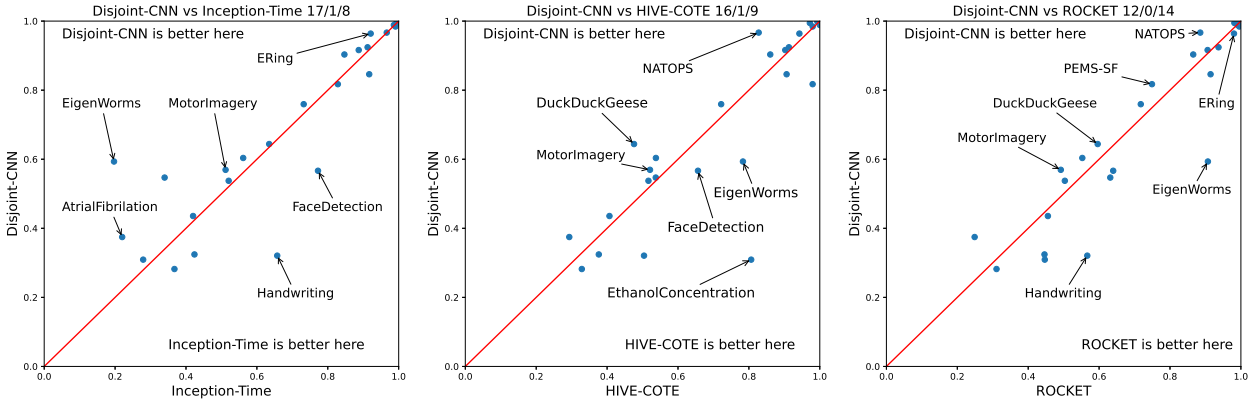
As shown in Fig. 7 we compare our model with the top 3 state of the arts. First, we compare Disjoint-CNN with the best deep learning model for MTSC, InceptionTime. Then we compare our method with the two non-deep learning state of the art models, ROCKET and HIVE-COTE. As illustrated in Fig. 7a, our model outperforms InceptionTime in 17 out of 26 datasets. Generally, our proposed model performs similarly to InceptionTime on datasets with high dimensions such as `DuckDuckGeese`, `PEMS-SF`. However, our model outperforms InceptionTime on datasets with longer time series length such as `EigenWorms`, `MotorImagery`. Besides, our Disjoint-CNN model is better at preventing overfitting on datasets with less training data like `AtrialFibrillation`, and `ERing`.

As shown in Fig. 7b 7c our proposed model outperforms HIVE-COTE and ROCKET on the time series with high dimensions like `DuckDuckGeese`, `PEMS-SF`, `MotorImagery`, and `NATOPS`. However, both HIVE-COTE and ROCKET outperforms Disjoint-CNN on datsasets with long time series length and low input dimensions such as `EigenWorms`, Indeed, as shown in Table II, all CNN based models fail to perform on the `EigenWorms` dataset. Note that Disjoint-CNN is the most accurate among all on this dataset. This is due to the limitation of CNN-based models which are unable to capture long term dependencies in the time series. We believe that adding LSTM layers would improve the performance and we leave it as future work. It is also interesting to observe from Table II that HIVE-COTE performs significantly better than all other methods on the `EthanolConcentration` dataset. This is because

Authorized licensed use limited to: UNIVERSIDAD DE GRANADA. Downloaded on March 16,2025 at 13:30:20 UTC from IEEE Xplore. Restrictions apply.

TABLE II: Average accuracies of ten algorithms over thirty re-samples datasets for the UEA MTSC archive

Model	Deep Learning-Based Models					Top 5 Non-Deep Learning-Based model				
Problem	Disjoin-CNN	InceptionTime	ResNet	TapNet	MLSTM	ROCKET	CIF	HIVE-COTE	STC	DTW_D
AWR	98.44	99.1	98.26	97.14	95.7	**99.56**	97.89	97.99	97.52	98.87
AF	**37.48**	22	36.23	30.23	26.7	24.89	25.12	29.34	31.78	23.56
BM	**100**	**100**	**100**	99.17	95.5	99	99.75	**100**	97.92	95.25
CR	99.91	99.45	99.4	97.5	91.7	**100**	98.38	99.31	98.94	100
DDG	**64.4**	63.47	63.2	58.27	46	59.6	56	47.6	43.47	49.2
EW	59.34	19.7	45.45	21	50.4	**90.69**	90.34	78.33	65.61	64.59
EP	98.87	98.65	99.18	96.09	73.2	99.09	98.39	**100**	98.64	96.31
EC	30.93	27.93	28.62	28.99	37.3	44.68	72.89	80.65	**82.42**	30.16
ER	96.39	92.1	87.19	89.46	87.26	**98.05**	95.66	94.25	84.29	92.92
FD	56.67	**77.24**	62.98	52.87	55	63.98	68.89	65.64	69.77	53.28
FM	**60.37**	56.14	54.7	51.34	55	55.27	53.9	53.77	53.51	54.17
HMD	32.44	42.39	35.32	32.35	36.5	44.6	**52.21**	37.66	34.39	30.32
HW	32.09	**65.74**	59.79	32.96	30.9	56.67	35.13	50.42	28.43	61.21
HB	75.94	73.21	63.89	73.97	66.3	71.76	**76.53**	72.18	50.74	68.88
LIB	91.62	88.73	**94.12**	83.63	85.3	90.62	91.67	90.26	84.47	88.04
LSST	54.7	33.97	42.94	46.34	52.8	**63.16**	56.18	53.73	27.54	54.76
MI	56.94	51.17	49.77	**59**	51	49.2	51.8	52.1	50.84	52.1
NATO	96.67	96.63	**97.12**	90.3	88.9	88.54	84.41	82.76	84.36	82.04
PD	99.47	**99.69**	99.65	93.66	97.8	98.17	98.97	97.2	97.71	99.28
PEMS	81.74	82.84	81.95	79.21	74.6	74.92	**99.85**	97.98	98.37	77.06
PS	28.21	**36.75**	30.86	17.5	11	31.07	26.56	32.87	30.63	15.39
RS	84.61	**91.69**	91.23	85.82	85.6	91.45	89.3	90.64	88.81	85.64
SRS1	**90.33**	84.69	76.11	85.4	84	86.56	85.94	86.03	84.72	81.81
SRS2	**53.76**	52.04	50.25	53.47	50.6	50.34	48.88	51.67	51.63	53.69
SWJ	43.56	42	30.89	35.12	26.7	**45.56**	45.12	40.67	44	22
UW	92.42	91.23	88.36	88.39	85.9	**93.69**	92.42	91.3	87.04	92.29
Average Rank	**3.86**	4.44	5.48	6.96	7.84	3.98	4.53	4.73	6.73	6.42



(a) (b) (c)

Fig. 7: Pairwise comparison of Disjoint-CNN model with the top deep model, Inception-Time, and two top non-deep models, ROCKET and HIVE-COTE.

Authorized licensed use limited to: UNIVERSIDAD DE GRANADA. Downloaded on March 16,2025 at 13:30:20 UTC from IEEE Xplore. Restrictions apply.

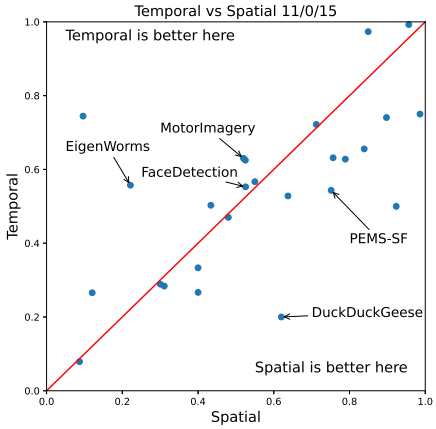


Fig. 8: Pairwise comparison of the accuracy of models when we use only temporal or spatial convolutions on MTSC benchmarking archive.

this dataset is based on the spectra of water-and-ethanol, hence interval and shapelet based approaches which are also components of HIVE-COTE performs better. This is indicated by the top 3 algorithms in Table II. On the other had, ROCKET has a few big wins compared to Disjoint-CNN (Fig 7c). Most of these datasets where ROCKET performs better, such as the `Handwriting` dataset have small number of time series instances per class. For instance `Handwriting` has 26 classes with 150 training instances, that is about 6 time series per class. This is insufficient to train our Disjoint-CNN to achieve better performance.

F. Component analysis

Finally, we studied the impact of each component of the Disjoint-CNN model. First, we investigated the impact of temporal and spatial convolutions, then used the 1+1D blocks in FCN and ResNet architectures to assess the performance of these models when the 1+1D is used instead of 1D convolutions.

1) Temporal vs Spatial convolution: In order to measure the impact of temporal and spatial filters on multivariate time series classification, we designed two networks with the same number of parameters. The first one used temporal convolution, and the other used spatial convolution to extract features from multivariate time series. Fig. 8 shows the pairwise comparison of these two networks applied on MTSC repository. As can be seen, in 15 out of 26 datasets, the spatial convolution can extract more discriminative patterns than temporal convolution. As depicted, Temporal network performs better on long lengths Series like `EigenWorms` and `MotorImagery` .Besides, the Spatial network also works well on high dimension ones, including `DuckDuckGeese` and `PEMS-SF`. Fig. 9 also demonstrates that the overall performance of the spatial network is better than the temporal one in terms of average ranking on the default test and train set of the UEA MTSC repository.

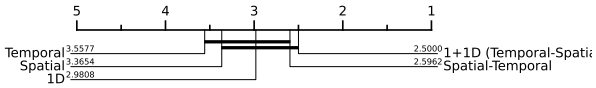


Fig. 9: Critical difference diagram for various combination of spatial and temporal convolutions.

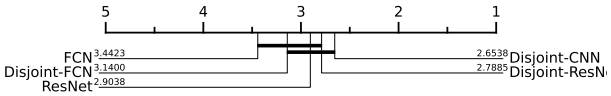


Fig. 10: Critical difference of typical and disjoint variants of FCN and ResNet models compared to our Disjoint model

2) Convolution factorization: Different combinations of spatial/temporal filters - including 1+1D, 1D and Spatial-Temporal (first applies spatial filters and then temporal) - were tested in order to investigate their effects on performance. As illustrated in Fig. 9, factorization improves the performance since the the spatial-temporal and 1+1D outperform the 1D convolution.

3) Disjoint-FCN and Disjoint-ResNet: Finally, to demonstrate that our 1+1D block works well on existing FCN and ResNet architectures, we replaced the typical convolution block in FCN and RsNet with 1+1D block. Fig. 10 summarizes the performance of typical and disjoint variants of FCN and ResNet models, where our 1+1D block is shown to improve the performance of these models. Notably, the performance of the model heavily depends on the baseline network. It can be seen that Disjoint-ResNet (2.78) and Disjoint-FCN (3.14) have outperformed typical ResNet (2.90) and typical FCN (3.44) in terms of average rank; although Disjoint-ResNet(2.78) has demonstrated a desirable performance, it is still inferior to our Disjoint-CNN model (2.65).

V. Conclusion and Future Work

In this paper, we propose a novel convolution filter block called 1+1D for the problem of multivariate time series classification, and we demonstrated that we can enhance the performance of convolution based deep learning MTSC models by incorporating such blocks in them. We then propose an effective MTSC method called Disjoint-CNN using our proposed 1+1D filter blocks. Through extensive experiments, we show that our model performs significantly better than state-of-the-art MTSC models on MTSC benchmarking repository datasets with the highest average rank among all existing MTSC models. We specifically found that our Disjoint-CNN model outperforms the state-of-the-art MTSC models in high dimensional settings and where we have large training data. In future, first we intend to apply Disjoint-CNN to big data sets (i.e., large number of time series with high dimensions). Second, we intend to focus on long time series such as sensor readings in human activity recognition application and extend our Disjoint-CNN model to be able to capture the temporal dependencies exist in such long time series.

Authorized licensed use limited to: UNIVERSIDAD DE GRANADA. Downloaded on March 16,2025 at 13:30:20 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] A. P. Ruiz, M. Flynn, J. Large, M. Middlehurst, and A. Bagnall, "The great multivariate time series classification bake off: a review and experimental evaluation of recent algorithmic advances," *Data Mining and Knowledge Discovery*, pp. 1–49, 2020.

[2] A. Bagnall, J. Lines, A. Bostrom, J. Large, and E. Keogh, "The great time series classification bake off: a review and experimental evaluation of recent algorithmic advances," *Data Mining and Knowledge Discovery*, vol. 31, no. 3, pp. 606–660, 2017.

[3] A. Bagnall, H. A. Dau, J. Lines, M. Flynn, J. Large, A. Bostrom, P. Southam, and E. Keogh, "The uea multivariate time series classification archive, 2018," *arXiv preprint arXiv:1811.00075*, 2018.

[4] H. A. Dau, A. Bagnall, K. Kamgar, C.-C. M. Yeh, Y. Zhu, S. Gharghabi, C. A. Ratanamahatana, and E. Keogh, "The ucr time series archive," *IEEE/CAA Journal of Automatica Sinica*, vol. 6, no. 6, pp. 1293–1305, 2019.

[5] C. W. Tan, F. Petitjean, and G. I. Webb, "Fastee: Fast ensembles of elastic distances for time series classification," *Data Mining and Knowledge Discovery*, vol. 34, no. 1, pp. 231–272, 2020.

[6] A. Dempster, F. Petitjean, and G. I. Webb, "Rocket: Exceptionally fast and accurate time series classification using random convolutional kernels," *arXiv preprint arXiv:1910.13051*, 2019.

[7] H. I. Fawaz, B. Lucas, G. Forestier, C. Pelletier, D. F. Schmidt, J. Weber, G. I. Webb, L. Idoumghar, P.-A. Muller, and F. Petitjean, "Inceptiontime: Finding alexnet for time series classification," *Data Mining and Knowledge Discovery*, vol. 34, no. 6, pp. 1936–1962, 2020.

[8] H. I. Fawaz, G. Forestier, J. Weber, L. Idoumghar, and P.-A. Muller, "Deep learning for time series classification: a review," *Data Min Knowl Discov*, vol. 33, no. 4, pp. 917–963, 2019.

[9] M. Middlehurst, J. Large, G. Cawley, and A. Bagnall, "The temporal dictionary ensemble (tde) classifier for time series classification," in *The European Conference on Machine Learning and Principles and Practice of Knowledge Discovery in Databases*, 2020.

[10] A. Shifaz, C. Pelletier, F. Petitjean, and G. I. Webb, "Ts-chief: a scalable and accurate forest algorithm for time series classification," *Data Mining and Knowledge Discovery*, vol. 34, no. 3, pp. 742–775, 2020.

[11] F. Karim, S. Majumdar, H. Darabi, and S. Harford, "Multivariate lstm-fcns for time series classification," *Neural Networks*, vol. 116, pp. 237–245, 2019.

[12] P. Schäfer and U. Leser, "Multivariate time series classification with weasel+ muse," *arXiv preprint arXiv:1711.11343*, 2017.

[13] I. Karlsson, P. Papapetrou, and H. Boström, "Generalized random shapelet forests," *Data mining and knowledge discovery*, vol. 30, no. 5, pp. 1053–1085, 2016.

[14] T. Le Nguyen, S. Gsponer, I. Ilie, M. O'Reilly, and G. Ifrim, "Interpretable time series classification using linear models and multi-resolution multi-domain symbolic representations," *Data mining and knowledge discovery*, vol. 33, no. 4, pp. 1183–1222, 2019.

[15] M. Middlehurst, J. Large, and T. Bagnall, "The canonical interval forest (CIF) classifier for time series classification," in *IEEE International Conference on Big Data*. IEEE Conference Publications, Oct. 2020.

[16] Y. Zheng, Q. Liu, E. Chen, Y. Ge, and J. L. Zhao, "Time series classification using multi-channels deep convolutional neural networks," in *International Conference on Web-Age Information Management*. Springer, 2014, pp. 298–310.

[17] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *nature*, vol. 521, no. 7553, pp. 436–444, 2015.

[18] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "Imagenet classification with deep convolutional neural networks," *Advances in neural information processing systems*, vol. 25, pp. 1097–1105, 2012.

[19] J. Schmidhuber, "Deep learning in neural networks: An overview," *Neural networks*, vol. 61, pp. 85–117, 2015.

[20] J. Sun, Y. Fu, S. Li, J. He, C. Xu, and L. Tan, "Sequential human activity recognition based on deep convolutional network and extreme learning machine using wearable sensors," *Journal of Sensors*, vol. 2018, 2018.

[21] X. Zhang, Y. Gao, J. Lin, and C.-T. Lu, "Tapnet: Multivariate time series classification with attentional prototypical network," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, no. 04, 2020, pp. 6845–6852.

[22] Z. Qiu, T. Yao, and T. Mei, "Learning spatio-temporal representation with pseudo-3d residual networks," in *proceedings of the IEEE International Conference on Computer Vision*, 2017, pp. 5533–5541.

[23] D. Tran, H. Wang, L. Torresani, J. Ray, Y. LeCun, and M. Paluri, "A closer look at spatiotemporal convolutions for action recognition," in *Proceedings of the IEEE conference on Computer Vision and Pattern Recognition*, 2018, pp. 6450–6459.

[24] M. Shokoohi-Yekta, B. Hu, H. Jin, J. Wang, and E. Keogh, "Generalizing dtw to the multi-dimensional case requires an adaptive approach," *Data mining and knowledge discovery*, vol. 31, no. 1, pp. 1–31, 2017.

[25] H. Deng, G. Runger, E. Tuv, and M. Vladimir, "A time series forest for classification and feature extraction," *Information Sciences*, vol. 239, pp. 142–153, 2013.

[26] C. H. Lubba, S. S. Sethi, P. Knaute, S. R. Schultz, B. D. Fulcher, and N. S. Jones, "catch22: Canonical time-series characteristics," *Data Mining and Knowledge Discovery*, vol. 33, no. 6, pp. 1821–1852, 2019.

[27] L. Ye and E. Keogh, "Time series shapelets: a novel technique that allows accurate, interpretable and fast classification," *Data mining and knowledge discovery*, vol. 22, no. 1, pp. 149–182, 2011.

[28] L. Breiman, "Random forests," *Machine learning*, vol. 45, no. 1, pp. 5–32, 2001.

[29] G. Ifrim and C. Wiuf, "Bounded coordinate-descent for biological sequence classification in high dimensional predictor space," in *Proceedings of the 17th ACM SIGKDD international conference on knowledge discovery and data mining*, 2011, pp. 708–716.

[30] T. Le Nguyen, S. Gsponer, and G. Ifrim, "Time series classification by sequence learning in all-subsequence space," in *2017 IEEE 33rd international conference on data engineering (ICDE)*. IEEE, 2017, pp. 947–958.

[31] J. Lin, E. Keogh, L. Wei, and S. Lonardi, "Experiencing sax: a novel symbolic representation of time series," *Data Mining and knowledge discovery*, vol. 15, no. 2, pp. 107–144, 2007.

[32] P. Schäfer and M. Högqvist, "Sfa: a symbolic fourier approximation and index for similarity search in high dimensional datasets," in *Proceedings of the 15th international conference on extending database technology*, 2012, pp. 516–527.

[33] A. Bagnall, M. Flynn, J. Large, J. Lines, and M. Middlehurst, "On the usage and performance of the hierarchical vote collective of transformation-based ensembles version 1.0 (hive-cote v1. 0)," in *International Workshop on Advanced Analytics and Learning on Temporal Data*. Springer, 2020, pp. 3–18.

[34] J. Hills, J. Lines, E. Baranauskas, J. Mapp, and A. Bagnall, "Classification of time series by shapelet transformation," *Data mining and knowledge discovery*, vol. 28, no. 4, pp. 851–881, 2014.

[35] M. Middlehurst, W. Vickers, and A. Bagnall, "Scalable dictionary classifiers for time series classification," in *International Conference on Intelligent Data Engineering and Automated Learning*. Springer, 2019, pp. 11–19.

[36] J. Large, J. Lines, and A. Bagnall, "A probabilistic classifier ensemble weighting scheme based on cross-validated accuracy estimates," *Data mining and knowledge discovery*, vol. 33, no. 6, pp. 1674–1709, 2019.

[37] Z. Wang, W. Yan, and T. Oates, "Time series classification from scratch with deep neural networks: A strong baseline," in *2017 International joint conference on neural networks (IJCNN)*. IEEE, 2017, pp. 1578–1585.

[38] F. Karim, S. Majumdar, H. Darabi, and S. Chen, "Lstm fully convolutional networks for time series classification," *IEEE access*, vol. 6, pp. 1662–1669, 2017.

[39] D.-A. Clevert, T. Unterthiner, and S. Hochreiter, "Fast and accurate deep network learning by exponential linear units (elus)," *arXiv preprint arXiv:1511.07289*, 2015.

[40] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," *arXiv preprint arXiv:1502.03167*, 2015.

[41] J. Demšar, "Statistical comparisons of classifiers over multiple data sets," *The Journal of Machine Learning Research*, vol. 7, pp. 1–30, 2006.

[42] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *arXiv preprint arXiv:1412.6980*, 2014.

Authorized licensed use limited to: UNIVERSIDAD DE GRANADA. Downloaded on March 16,2025 at 13:30:20 UTC from IEEE Xplore. Restrictions apply.